

Kursy online

Igor Zamojski

Projekt zaliczeniowy - UJ 2026

Projektowanie Aplikacji Internetowych

Kursy online

Jedna platforma do wszystkich funkcjonalności kursów

- Strony sprzedażowe
- Wykłady
- Quizy
- Q&A
- Zadania domowe
- Egzaminy
- Certyfikaty

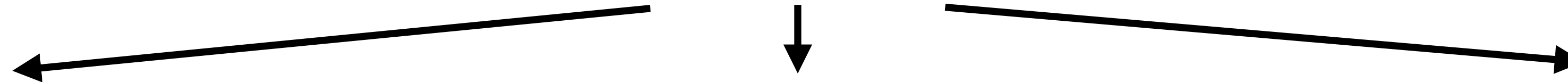
Kursy online

Jedna platforma do wszystkich funkcjonalności kursów

Jedna platforma z odpowiednio rozdzielonymi obowiązkami



sysadmin



school admin



instructors



course participants



school admin



instructors



course participants



school admin



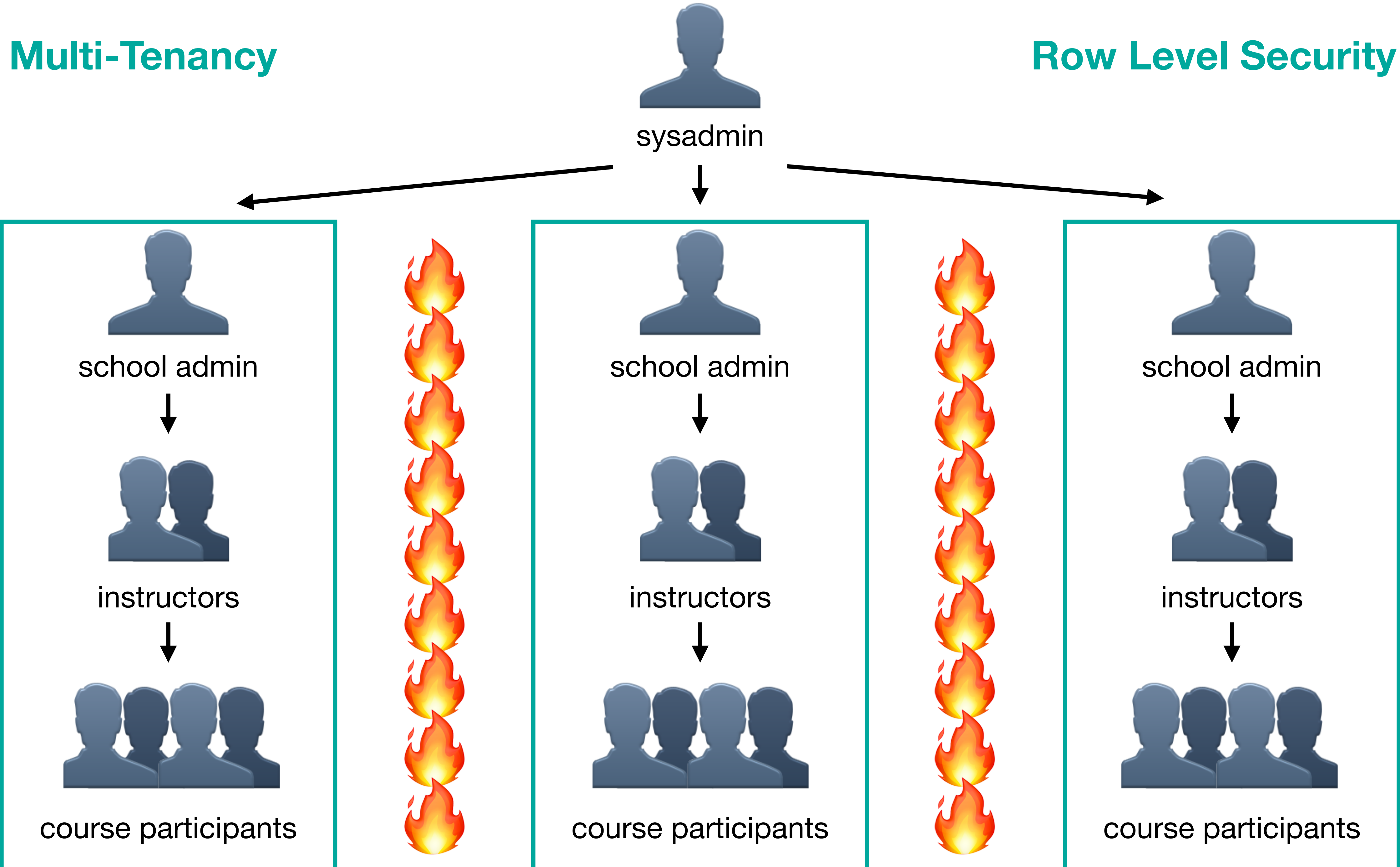
instructors

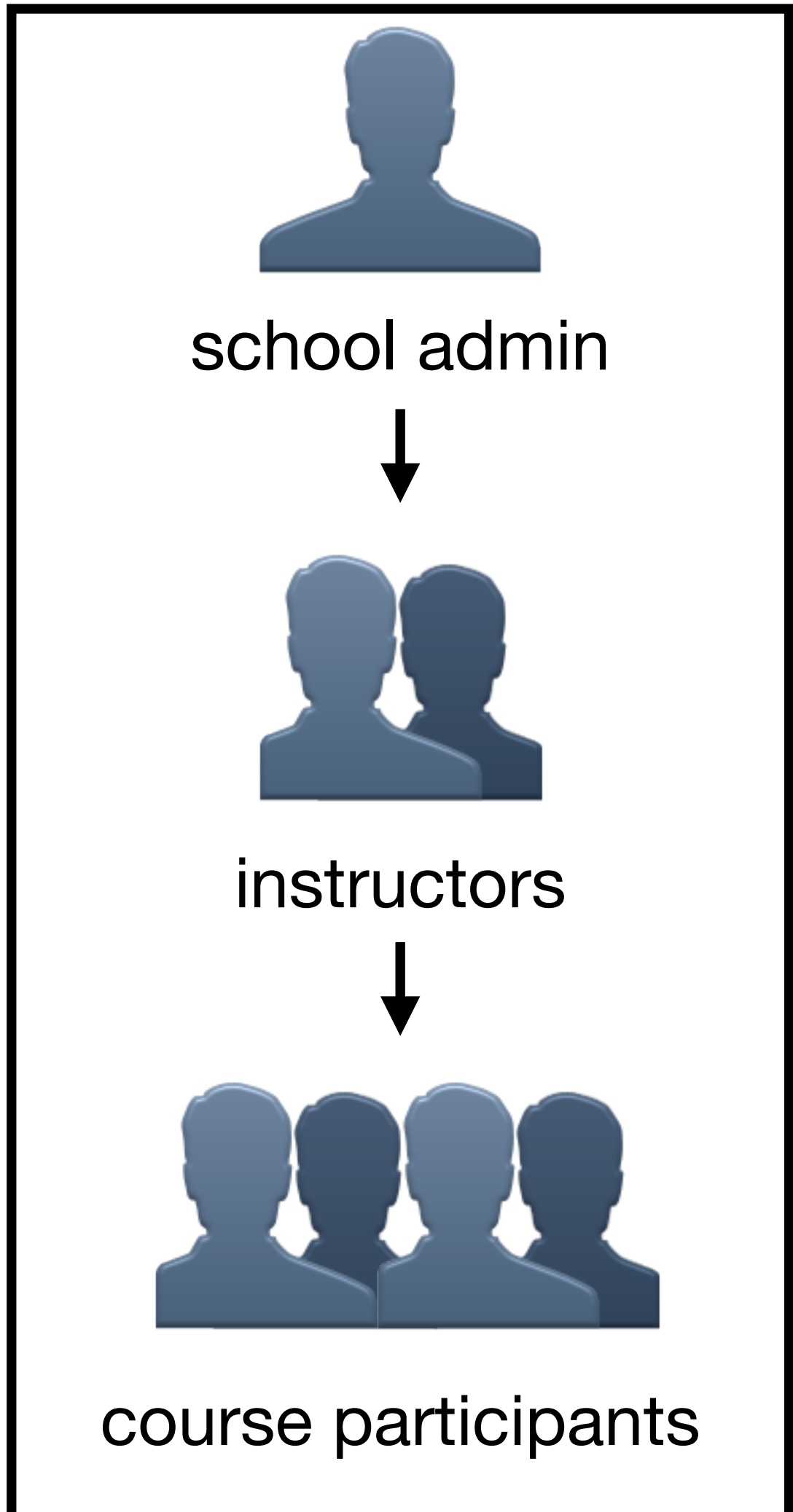
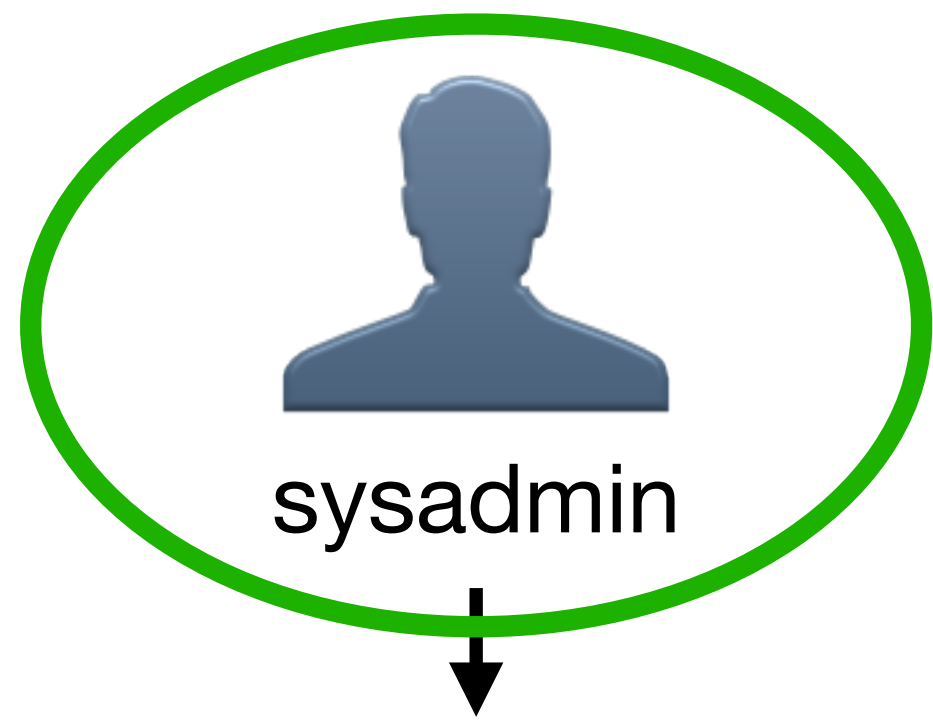


course participants

Multi-Tenancy

Row Level Security





NGV

SYSTEM

Schools

Plans

Back to Schools

Demo School

demo-school1

Enterprise

Active

COURSES

2

MEMBERS

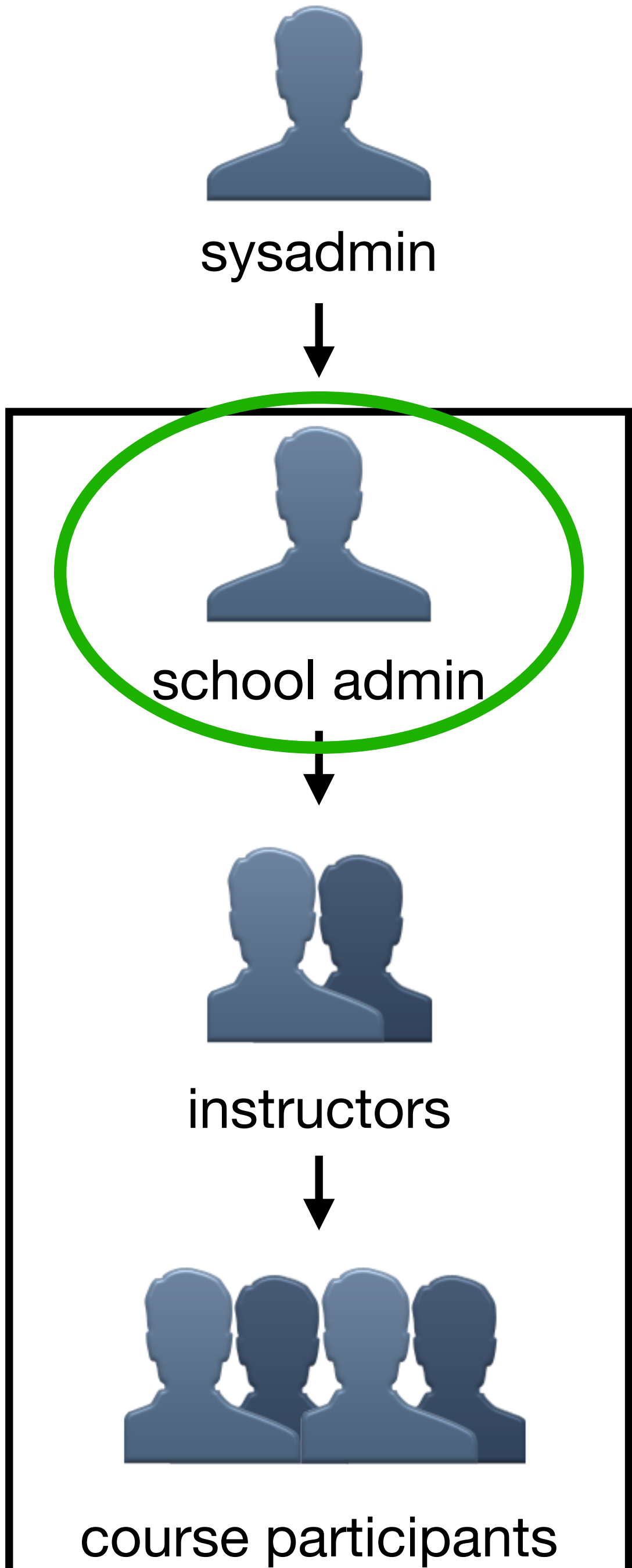
5

ENROLLMENTS

3

MEMBERS (5)

Name	Email	Role	Joined
Anna Admin	a@e.com	School_admin	Jun 14, 2026
Ivan Instructor	i@e.com	Instructor	Jun 14, 2026
Sara Student	s1@e.com	Student	Jun 14, 2026
Sam Student	s2@e.com	Student	Jun 14, 2026
Sofia Student	s3@e.com	Student	Jun 14, 2026



N

NGV

ADMIN

Dashboard

Courses2

Instructors

Students

Enrollments

Translations

Curriculum

Students

Reviews2

Settings

Review Queue

Pending instructor submissions waiting for approval.

2 PENDING REVIEWS

Interfaces vs Type Aliases

Ivan Instructor · 13m ago

Reject

Approve

CURRENT (PUBLISHED)	PROPOSED CHANGES (4 CHANGES)
TEXT TypeScript interfaces define the shape of an object.	TEXT TypeScript interfaces define the shape of an object and support declaration merging.
TEXT Type aliases can represent any type, not just objects.	TEXT Type aliases can represent any type – primitives, unions, intersections, and objects.
	TEXT Rule of thumb: prefer interface for object shapes; use type for everything else.
	VIDEO Interfaces vs Type Aliases – overview – https://example.com/videos/interfaces-vs-types.mp4

Utility Types: Partial & Required

Ivan Instructor · 13m ago

Reject

Approve



sysadmin



school admin



instructors



course participants

N

NGV

CoursesDashboard

SSSofia Student

Progress100%

Types & Interfaces2/2 lessons

What is TypeScript?

Primitive Types

Advanced Types7/7 lessons

Union & Intersection Types

Discriminated Unions

Mapped Types

Conditional Types

Template Literal Types

Advanced Types Quiz

Advanced Types Homework

Functions & Classes6/6 lessons

Typing Functions

Overloads & Rest Parameters

Classes & Access Modifiers

Abstract Classes & Implements

Functions & Classes Quiz

Functions & Classes Homework

CONTENT

Conditional Types

Conditional Types

Conditional types select one of two possible types based on a condition. The syntax mirrors a ternary expression: `T extends U ? X : Y`.`

```
type IsArray<T> = T extends any[] ? true : false;

type A = IsArray<string[]>; // true
type B = IsArray<number>;   // false
```

infer

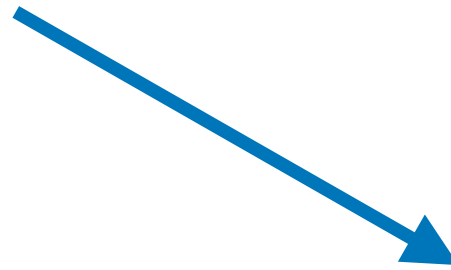
The `infer` keyword inside the extends clause lets you capture the matched type into a variable:

```
type UnpackArray<T> = T extends (infer U)[] ? U : T;

type Unpacked = UnpackArray<string[]>; // string
```

Mark as Complete

SEO, ISR



Strony sprzedażowe

Wykłady



Video processing

Interactive UI



Quizy

Q&A

Zadania domowe

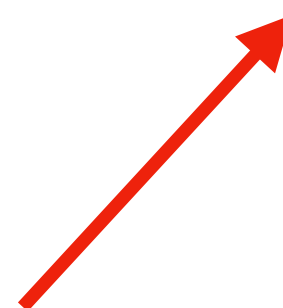
Egzaminy



Certyfikaty

Email handling

Task queue



ADR

1/8 : Język

Decyzja

Node.js + TypeScript – jeden język dla backendu i frontendu.

Rozważane alternatywy

PHP (Laravel)

- Bardzo popularny, dojrzały ekosystem, wbudowane migracje i auth.
- PHP + JavaScript.

Django (Python)

- Wbudowany admin.
- Python + JavaScript.
- Dobry do przetwarzania danych, ale nie do bogatego interaktywnego UI.

Spring (Java)

- Dojrzały, silnie typowany, dobry do bardzo dużych systemów.
- Duży narzut konfiguracyjny (klasy, adnotacje, dependency injection) nieproporcjonalny do skali.
- Java + JavaScript.

Node.js + TypeScript

Uzasadnienie

- Projekt wymaga bogatego UI, JavaScript w przeglądarce jest nieunikniony niezależnie od backendu.
 - Node.js eliminuje podwójny ekosystem: typy żądań/odpowiedzi API i schematy walidacji definiujemy raz i importujemy po obu stronach.
 - TypeScript po obu stronach oznacza, że kompilator wykrywa rozbieżności między backendem a frontendem w czasie kompilacji.
-

Trade-offy

- Node.js jest jednowątkowy, zadania obliczeniowe (PDF, transkodowanie) blokują pętlę zdarzeń i wymagają osobnego procesu.
- TypeScript wymaga dodatkowej konfiguracji i dyscypliny – błędy typów itd.

2 / 8 : Framework

Decyzja

Next.js 14 z App Router – jeden projekt obsługuje SSR, incremental static regeneration, komponenty klienckie i API.

Rozważane alternatywy

Remix

- Zbudowany na React, dobre wsparcie dla zagnieżdżonych layoutów i formularzy.
- Brak ISR, każda strona renderowana przy każdym żądaniu, co jest kosztowne dla stron kursów z dużym ruchem.
- Mniejszy ekosystem i mniejsza społeczność niż Next.js.

Czysta aplikacja React + Express

- Express minimalistyczny i popularny, React bez frameworka (np. z Vite) dobrze znany.
- Dwa osobne projekty backend+frontend.
- Publiczne strony kursów byłyby single page application – wyszukiwarki nie indeksują treści poprawnie, a to jest ważne dla SEO.

Next.js + App Router

Uzasadnienie

- Publiczne strony kursów potrzebują incremental static regeneration (`revalidate = 60`) – Next.js obsługuje to natywnie, pozostałe opcje nie.
 - App Router pozwala mieszać komponenty serwerowe (bezpośrednie zapytania do DB) z klienckimi w jednym drzewie, nie trzeba duplikować logiki pobierania danych.
 - Next.js obsługuje jednocześnie SSR, ISR i Route Handlers API.
-

Trade-offy

- Granica między komponentem serwerowym a klienckim (`'use client'`) wymaga świadomego zarządzania.

3/8 : API

Next.js Route Handlers

Decyzja

Next.js Route Handlers w tym samym repozytorium co frontend.

Rozważane alternatywy

Osobny backend Express

- Bardzo popularny, minimalistyczny, duża społeczność.
- Dwa wdrożenia, dwie konfiguracje, zduplikowane typy TypeScript.
- Strony SSR muszą odpytywać API przez sieć zamiast bezpośrednio.

tRPC

- End-to-end type safety bez generowania kodu, świetny DX w TypeScript.
- Nie obsługuje REST, zewnętrzne systemy integrujące się przez HTTP nie mogą go używać.
- Wymaganie publicznego API z kluczami dostępu wyklucza to rozwiązanie.

Uzasadnienie

- Projekt wymaga publicznego REST API dostępnego z zewnętrznych systemów..
 - Route Handlers w Next.js współdzielą typy z frontendem bezpośrednio, bez etapu generowania kodu.
 - Komponenty serwerowe mogą odpytywać DB bezpośrednio zamiast przez dodatkowe zapytanie HTTP.
 - Jeden serwis Docker uruchamia cały stack.
-

Trade-offy

- Struktura API jest powiązana ze strukturą folderów. Refaktoryzacja oznacza fizyczne przenoszenie folderów, a nie zmianę jednej linijki w definicji routera.
- Route Handlers mogą urosnąć, jeśli kompozycja middleware nie jest pilnowana.

4 / 8 : Baza danych

Rozważane alternatywy

MySQL / MariaDB

- Szeroko wdrażany.
- Brak Row-Level Security - izolację można wymuszać tylko w kodzie aplikacji, co jest podatne na błędy.

MongoDB

- Naturalnie obsługuje dokumenty o zmiennym schemacie (dobre dla np. bloków lekcji).
- Brak złączeń - pobranie kursu z modułami, lekcjami i zapisami wymaga wielu zapytań lub denormalizacji.
- Dane relacyjne (zapisy, role, uprawnienia) słabo pasują do modelu dokumentowego.

Drizzle (alternatywa dla Prisma)

- TypeScript-first, składnia zbliżona do SQL.
- Mniej dojrzały ekosystem.

PostgreSQL + Prisma

Uzasadnienie

- PostgreSQL jako jedyna popularna baza obsługuje Row-Level Security.
- Prisma generuje typy TypeScript ze schematu automatycznie.
- `prisma migrate dev` generuje SQL migracji i zaktualizowany klient TypeScript jedną komendą.

Trade-offy

- Prisma nie obsługuje RLS natywnie, polityki bezpieczeństwa muszą być pisane w surowym SQL w plikach migracji.

5 / 8 : Uwierzytelnianie

Decyzja

Własne JWT z krótkotrwałymi tokenami dostępu (15 min, w pamięci klienta) i długotrwałymi tokenami odświeżania (30 dni, cookie HTTP-only). Lista zablokowanych tokenów w Redis. Klucze API jako osobna ścieżka - hashe w DB.

Rozważane alternatywy

NextAuth.js (Auth.js)

- Najpopularniejsza biblioteka auth dla Next.js, obsługuje OAuth i sesje JWT.
- Dodanie `schoolId`, `role`, `jti` w payloadzie wymaga walki z konwencjami biblioteki.

JWT + Redis blocklist

Uzasadnienie

- Niestandardowy payload JWT: `userId`, `schoolId`, `role`, `jti` i wymaganie odwoływalności sprawiają, że własna implementacja jest prostsza niż adaptacja NextAuth.
- Redis planowany do zarządzania tokenami współdzieli infrastrukturę z kolejką zadań.
- Payload JWT zawiera cały kontekst (`userId`, `schoolId`, `role`) - każde żądanie autentykowane bez zapytania do DB.
- Cookie HTTP-only dla refresh tokena chroni przed XSS.
- Wylogowanie jest natychmiastowe - JTI trafia na listę zablokowanych w Redis.

Trade-offy

- Więcej kodu do utrzymania niż przy bibliotece.
- Logika odświeżania tokenu musi być zaimplementowana po stronie klienta; błąd w niej powoduje niespodziewane wylogowania.
- Lista zablokowanych JTI rośnie - wymaga zadania czyszczącego w tle (rozwiązanie: TTL Redisa).

6/8 : Kolejka zadań

BullMQ + Redis

Decyzja

BullMQ jako biblioteka kolejki zadań na Redis. Osobny proces roboczy jako drugi serwis Docker. Klienci odpytują API `GET jobs/:jobId`.

Rozważane alternatywy

PostgreSQL jako kolejka (tabela z odpytywaniem)

- Brak dodatkowej infrastruktury - PostgreSQL jest wymagany przez aplikację.
- Regularne odpytywanie tabeli generuje niepotrzebne obciążenie DB przy wzroście liczby zadań.

Worker Threads Node.js

- Brak zewnętrznych zależności.
- Zadania nie są utrwalane, restart procesu traci wszystkie zadania w toku.

Uzasadnienie

- Generowanie PDF i transkodowanie wideo wymagają trwałości zadań i automatycznych prób.
- Redis jest już planowany do zarządzania tokenami JWT - BullMQ nie dodaje nowej infrastruktury.
- Wzorzec polling (`GET /jobs/:jobId`) jest wystarczający dla czasów zadań rzędu sekund.
- BullMQ automatycznie ponawia nieudane zadania z wykładniczym backoffem.
- Awaria workera nie wpływa na serwer webowy - jest to osobny proces.

Trade-offy

- Stack wymaga dwóch uruchomionych procesów: serwer Next.js i worker.
- Docker Compose staje się bardziej złożony; dwa serwisy współdzielą połączenia do Redis i DB.

7 / 8 : Walidacja

Zod + resolver

Decyzja

Zod - schematy definiowane raz, importowane przez backend i frontend. Typy TypeScript wyprowadzane z schematu.

Rozważane alternatywy

class-validator

- Wymaga instancji klasy do walidacji, co nie pasuje do funkcyjnego stylu Next.js Route Handlers i React.

Ręczna walidacja (if/throw)

- Brak zależności.
- Każdy endpoint ma własne sprawdzeni; niespójne kształty błędów, liniowy koszt utrzymania.

Uzasadnienie

- Zod wyprowadza typy TypeScript bezpośrednio ze schematu - jeden obiekt jest i schematem walidacji, i definicją typu.
 - Resolver sprawia, że błędy walidacji formularza są identyczne z błędami API.
 - Schemat to jedyne źródło prawdy dla kształtu żądania.
 - Zmiana reguły walidacji w jednym miejscu automatycznie działa po obu stronach.
-

Trade-offy

- Złożone warunkowe reguły (np. A wymagane tylko gdy B = X) stają się rozwlekłe w składni Zod.

8 / 8 : Storage

MinIO_(dev) / AWS S3_(prod)

Decyzja

MinIO (dev, Docker) i AWS S3 (produkcja) - to samo API S3. Pliki przesyłane bezpośrednio z przeglądarki przez **presigned URL**. Certyfikaty: prywatne, serwowane przez krótkotrwałe presigned URL do pobierania.

Rozważane alternatywy

Lokalny system plików serwera

- Zero konfiguracji, prosta implementacja.
- Pliki tracone przy zastępowaniu kontenera.
- Nie działa przy wielu instancjach za load balancerem.

Uzasadnienie

- Presigned URL eliminuje serwer z ścieżki przesyłania; duże pliki wideo trafiają bezpośrednio z przeglądarki do storage.
 - MinIO implementuje to samo API co S3 - przełączenie na produkcję to TYLKO zmiana zmiennych środowiskowych.
 - Certyfikaty PDF są domyślnie prywatne - student dostaje presigned URL wygasający po krótkim czasie.
-

Trade-offy

- Upload wymaga dwóch kroków: żądanie presigned URL z serwera, potem upload bezpośrednio do storage.

Dziękuję. Czy są jakieś pytania?

Igor Zamojski

Projekt zaliczeniowy - UJ 2026

Projektowanie Aplikacji Internetowych

Strony sprzedażowe
Wykłady
Quizy
Q&A
Zadania domowe
Egzaminy
Certyfikaty

